

Lion Guardians Patrols Effort

Technical Guide

Patrol Effort Analysis — Methodology & Calculation Reference
Version 1.0 · Generated July 21, 2026

1. Overview

The **LG Patrols Effort** workflow analyses ranger patrol activity in the Amboseli ecosystem for the **Lion Guardians** programme. It ingests patrol tracks and patrol events from **EarthRanger**, computes patrol effort maps and summary statistics per guardian and patrol type, and delivers an interactive dashboard plus a print-ready Word report.

The workflow produces three maps per group (Patrol Events, Patrol Trajectories, Linear Time Density), five scalar metrics, two charts (bar chart and pie chart), and six summary CSV tables.

Note: All per-group outputs are produced by iterating over a user-chosen grouper: patrol type, patrol serial number, or patrol subject.

2. Dependencies & Prerequisites

2.1 EarthRanger Connection

All patrol data is fetched from an **EarthRanger** instance via `set_er_connection`. The workflow uses `set_patrols_and_patrol_events_params` to configure the query, then calls `get_patrols_from_combined_params` and `get_patrol_observations_from_patrols_df_and_combined_params` to retrieve patrol tracks, and `unpack_events_from_patrols_df_and_combined_params` for events. The sub-page size is 200 records; patrols are not filtered to overlap the date range only.

2.2 Groupers

Four grouper fields are available via `set_groupers` (default: none, a single combined view):

Grouper field	Source column	Typical use
patrol_type	extra__patrol_type__value	One output per patrol type
patrol_serial_number	extra__patrol_serial_number	One output per patrol serial
patrol_status	extra__patrol_status	One output per patrol status
patrol_subject	extra__patrol_subject	One output per guardian ranger

2.3 Study Area Layers

Unlike some sibling Lion Guardians workflows, the study-area boundaries here are **not** downloaded from Dropbox — they are fetched live from the connected **EarthRanger** instance via `get_spatial_features`, querying two feature types:

Feature type	Style	Purpose
Conservancies	Fill #8fbc8b, 75% opacity, 1.75 px stroke	Conservancy boundary polygons
Group Ranch Boundaries	Unfilled, black 1.25 px stroke outline	Community ranch boundary polygons

There is no Dropbox-downloaded boundary file and no separate conflict-hotspot layer in this workflow — only the org logo and the two Word templates below come from Dropbox.

2.4 Word Document Templates & Logo

File	Purpose
patrol_guardians_cover_page.docx	Report cover page template — period, preparer
custom_patrol_template.docx	Per-grouper section template — maps, charts, and summary tables
lion-guardians.png	Organisation logo, embedded on the cover page

2.5 Base Map Tile Layers

Layer	Opacity	Max zoom
ArcGIS World Hillshade	100 %	20
ArcGIS World Street Map	15 %	20

The hillshade provides full-opacity terrain context. The street map is overlaid at 15 % to show roads and settlement names without obscuring terrain or patrol data.

3. Data Ingestion Pipeline

3.1 Patrol Observations → Relocations → Trajectories

`process_relocations` converts raw patrol observations to a standardised `GeoDataFrame`. Retained columns include patrol identifiers (`patrol_id`, `patrol_start_time`, `patrol_end_time`, `patrol_serial_number`, `patrol_subject`), fix quality (`junk_status`), and point geometry. Three null-island coordinate pairs are filtered out:

- (180.0, 90.0) — boundary sentinel
- (0.0, 0.0) — null-island artefact
- (1.0, 1.0) — common default / test value

A daytime filter (`filter_daytime_patrols`) is applied before trajectory construction, retaining only fixes between 06:00 and 19:00 local time to exclude night-time GPS drift. `relocations_to_trajectory` then connects consecutive fixes per patrol into `LineString` segments, adding `dist_meters`, `speed_kmhr`, `segment_start`, and `segment_end`. The following segment filter is applied:

Filter parameter	Default	Description
<code>min_length_meters</code>	10	Discard segments shorter than 10 m
<code>max_length_meters</code>	100 000	Discard segments longer than 100 km
<code>min_time_secs</code>	10	Discard segments shorter than 10 s
<code>max_time_secs</code>	21 600	Discard segments longer than 6 hours
<code>min_speed_kmhr</code>	1	Discard segments below 1 km/h average speed
<code>max_speed_kmhr</code>	7	Discard segments above 7 km/h average speed

Trajectories are persisted as `trajectories.geoparquet`.

3.2 Patrol Events

`unpack_events_from_patrols_df_and_combined_params` extracts associated events for each patrol. `get_event_type_display_names_from_events` enriches the events `GeoDataFrame` with human-readable event type

names (append_category_names: duplicates). Events are converted to the user timezone and persisted as events.geoparquet.

3.3 Temporal Index & Column Renaming

add_temporal_index keys the trajectory GeoDataFrame to extra__patrol_start_time, grouped by the configured groupers, enabling per-group iteration. Four columns are then renamed via map_columns (raise_if_not_found: true):

Original column	Renamed to
extra__patrol_type__value	patrol_type
extra__patrol_serial_number	patrol_serial_number
extra__patrol_status	patrol_status
extra__patrol_subject	patrol_subject

The renamed GeoDataFrame is split into per-group partitions by split_groups. All downstream map and metric tasks iterate over these partitions via mapvalues.

3.4 Trajectory Colour & Event Colormap

Patrol trajectories are **not** coloured by a user-selectable field — every track segment on the Trajectories map uses a single fixed colour (#008b8b, dark cyan), reflected in the map legend as a static “Foot patrols” entry. Patrol events, by contrast, **are** colour-mapped: apply_color_map maps event_type to the **Accent** colormap, writing the result to event_type_colormap, which drives both the Patrol Events map and the bar/pie charts.

4. Static Map Layers

Two static layers, fetched live from EarthRanger (see §2.3), are built once and composited onto every group-level map to provide spatial context.

4.1 Layer Styles

Layer	Colour	Opacity	Filled	Notes
Conservancies	#8fbc8b (dark sea green)	75 %	Yes	Stroke width 1.75 px, same colour as fill
Group Ranch Boundaries	Black outline	0 % fill	No	Outline only, stroke width 1.25 px

5. Map Outputs — Methodology

5.1 Patrol Events Map

create_scatterplot_layer renders each patrol event as a point marker, coloured by event_type_colormap (Accent colormap). Point radius is 2.5 px at 75 % opacity with stroked, black outlines. Before rendering, geometric outliers are removed via exclude_geom_outliers (z-threshold: 3) and null geometries are dropped. The event layer is combined with the two study-area static layers (§4.1). The map is auto-zoomed to the event extent and persisted as HTML (suffix: events), then converted to PNG at 2× scale with a 40 s tile-load wait.

5.2 Patrol Trajectories Map

`create_path_layer` renders patrol track segments with a fixed style (not driven by any colormap):

Property	Value
Colour	#008b8b, dark cyan (fixed for all patrols)
Width	2.25 px, min 2 px, max 8 px (screen-space pixels)
Cap / Join	Rounded
Opacity	45 %

The path layer is combined with the two study-area static layers and auto-zoomed to the trajectory extent. The map is persisted as HTML (suffix: `patrol_trajectories`) and, like the other maps, converted to PNG at 2× scale with a 40 s tile-load wait — PNG generation is active for this map, not disabled.

5.3 Linear Time Density Map

`create_meshgrid` builds a raster grid over the full patrol trajectory extent. `calculate_linear_time_density` then computes the time-weighted density of patrol coverage. Parameters:

Parameter	Value	Meaning
percentiles	50, 60, 70, 80, 90, 100	Contour probability thresholds extracted as polygons
intersecting_only	false	Meshgrid covers the full AOI extent

NaN-percentile rows are dropped and the result is sorted ascending. Contour polygons are coloured with the **RdYlGn** diverging colormap (innermost 50th percentile = red, outermost 100th = green) at 45 % opacity. The map is persisted as HTML (suffix: `time_density`) and converted to PNG at 2× scale with a 40 s tile-load wait.

6. Summary Metrics

6.1 Per-Guardian Statistics

`summarize_df` aggregates trajectory data grouped by `patrol_subject`:

Output column	Source column	Aggregator	Output unit
<code>no_of_patrols</code>	<code>extra__patrol_id</code>	nunique	count
<code>total_distance</code>	<code>dist_meters</code>	sum	km
<code>total_time</code>	<code>timespan_seconds</code>	sum	h

A separate summary counts the number of events per guardian (id nunique). Results are persisted as CSVs. A pivot table is also generated — events pivoted by `event_type_display` — to show each guardian's event breakdown.

6.2 Patrol Type Summary

The same three aggregations (`no_of_patrols`, `total_distance`, `total_time`) are also computed grouped by `patrol_type` (`summarized_patrol_types`).

Note: This table is computed by the workflow but its output is never persisted or wired into the dashboard or Word report — it is currently dead code in `spec.yaml`.

6.3 Event Type Summary

Events are summarised by `event_type_display` using `id_nunique`, giving a count of distinct events per type across the analysis period.

6.4 Monthly Summary

`decompose_datetime` extracts `month_name` from `extra__patrol_start_time` (prefixed `time_`). Patrol effort is then summarised by `time_month_name` (`no_of_patrols`, `total_distance`, `total_time`) to reveal seasonal patrol patterns.

6.5 Scalar Dashboard Widgets

Five scalar metrics are computed per group from the trajectory data and displayed as single-value widgets (1 decimal place):

Widget title	Source column	Aggregator	Unit
Total Patrols	<code>extra__patrol_id</code>	<code>nunique</code>	count
Total Time	<code>timespan_seconds</code>	<code>sum</code>	h
Total Distance	<code>dist_meters</code>	<code>sum</code>	km
Average Speed	<code>speed_kmhr</code>	<code>mean</code>	km/h
Max Speed	<code>speed_kmhr</code>	<code>max</code>	km/h

7. Event Charts

7.1 Time Series Bar Chart

`draw_time_series_bar_chart` plots patrol event counts over time. X-axis: time (event timestamp); Y-axis: `event_type_display`; aggregation: `count`; category colour: `event_type_colormap` (Accent colormap). The chart is persisted as HTML (suffix: `patrol_events_time_series_bar_chart`) and converted to PNG.

7.2 Pie Chart

`draw_pie_chart` shows the proportional breakdown of events by `event_type_display`, using `event_type_colormap` for slice colours and displaying raw counts as text labels. The chart is persisted as HTML (suffix: `patrols_pie_chart`) and converted to PNG.

8. Word Report (.docx)

8.1 Cover Page

`prepare_cover_metadata` builds the cover context (org logo, report period, *Ecoscope* as preparer, generation timestamp). `create_context_page` renders it into `cover_page.docx` using the `patrol_guardians_cover_page.docx` template.

8.2 Per-Grouper Sections

`create_guardians_context` assembles a context dict per group containing maps (events map, trajectories map, time density map), charts (pie chart, bar chart), and CSV data (monthly efforts, guardian event pivot, guardian events,

guardian patrol stats, event type efforts). `render_docx_page` (from `ecoscope_workflows_ext_lion_guardians`) renders each section from the `custom_patrol_template.docx` template. Image boxes: **9.779 × 16.4592 cm** ($\approx 3.85 \times 6.48$ in). `strict_images: true` catches missing PNGs before rendering.

8.3 Document Merge

`merge_docx_documents` concatenates the cover page (`cover_page.docx`) and all per-grouper sections, ordered by name, into a single Word file: `overall_report.docx`.

9. Interactive Dashboard

`gather_dashboard` assembles the patrol dashboard from ten widget groups:

Widget	Type	Source task
Total Distance	Single value	<code>total_patrol_dist</code> → <code>patrol_dist_grouped_widget</code>
Average Speed	Single value	<code>avg_speed</code> → <code>avg_speed_grouped_widget</code>
Max Speed	Single value	<code>max_speed</code> → <code>max_speed_grouped_widget</code>
Total Patrols	Single value	<code>total_patrols</code> → <code>total_patrols_grouped_sv_widget</code>
Total Time	Single value	<code>total_patrol_time</code> → <code>patrol_time_grouped_widget</code>
Patrol Events Map	Map	<code>draw_events</code> → <code>events_grouped_map_widget</code>
Trajectories Map	Map	<code>trajs_ecomap</code> → <code>trajs_grouped_map_widget</code>
Events Bar Chart	Plot	<code>patrol_events_bar_chart</code> → <code>grouped_bar_plot_widget_merge</code>
Events Pie Chart	Plot	<code>patrol_events_pie_chart</code> → <code>patrol_events_pie_widget_grouped</code>
Time Density Map	Map	<code>td_ecomap</code> → <code>td_grouped_map_widget</code>

Note: Widget tasks use `skipif: [never]` so the dashboard always assembles, even when some groups have no data.

10. Output Files

All files are written to `$ECOSCOPE_WORKFLOWS_RESULTS`.

File / pattern	Format	Content
<code>events.geoparquet</code>	GeoParquet	Patrol events with display names and timezone-converted timestamps
<code>trajectories.geoparquet</code>	GeoParquet	Patrol segments with <code>speed_kmhr</code> , <code>dist_meters</code>
<code>_events.html</code>	HTML	Interactive patrol events map
<code>_patrol_trajectories.html</code>	HTML	Interactive patrol trajectories map
<code>_time_density.html</code>	HTML	Interactive linear time density map
<code>_events.png</code>	PNG	2× screenshot of patrol events map

File / pattern	Format	Content
_time_density.png	PNG	2× screenshot of time density map
_patrols_pie_chart.png	PNG	2× screenshot of event type pie chart
_patrol_events_time_series_bar_chart.png	PNG	2× screenshot of events time series bar chart
_guardian_patrol.csv	CSV	Per-guardian patrol effort (patrols, distance, time)
_guardian_events.csv	CSV	Per-guardian event count
_pivot_guardian_events.csv	CSV	Guardian × event type pivot table
_event_types.csv	CSV	Per-event-type count
_monthly_patrol_efforts.csv	CSV	Monthly patrol effort summary
cover_page.docx	Word	Rendered report cover page
.docx	Word	Per-grouper report section
overall_report.docx	Word	Final combined Word report

11. Workflow Execution Logic

11.1 Skip Conditions

Two default skip conditions apply to every task (task-instance-defaults):

- **any_is_empty_df** — skips the task (and all dependants) when any input DataFrame is empty, handling patrol types or periods with no data gracefully.
- **any_dependency_skipped** — propagates skips downstream automatically.

Widget and map-widget tasks override this with skipif: [never] to ensure the dashboard always assembles.

11.2 Data Flow Summary

Stage	Tasks
Setup	ER connection, time range, timezone, groupers, base maps
Study area	Conservancies + Group Ranch Boundaries fetched live from EarthRanger
Downloads	2 Word templates + org logo from Dropbox
Patrol ingest	Params → prefetch → observations → events → rename → convert TZ
Trajectories	Relocations → trajectories → temporal index → rename → split groups
Events branch	Filter → temporal index → colormap → rename → outlier removal → scatter layer → map → HTML → PNG → widget
Trajectories branch	Speed format → rename → colormap → path layer → map → HTML → widget
Time density branch	Meshgrid → LTD → drop NaN → sort → colormap → GeoJSON layer → map → HTML → PNG → widget

Stage	Tasks
Metrics branch	5 scalar widgets; 5 CSV summaries (guardian patrol, guardian events, event type, monthly, pivot)
Charts branch	Time series bar chart + pie chart → HTML → PNG → widgets
Report assembly	Cover page + per-group sections → merge docx
Dashboard	gather_dashboard combines all 10 widgets

12. Software Versions

Package	Version	Role
ecoscope-platform	>=2.15.0, <2.16.0	Consolidated core task library and workflow engine
ecoscope-workflows-ext-custom	0.1.0rc14.*	Utility tasks (maps, layers, column mapping)
ecoscope-workflows-ext-ste	0.0.0rc1.*	Spatial operations tasks (EarthRanger features, view state)
ecoscope-workflows-ext-lion-guardians	0.0.0rc1.*	Lion Guardians domain tasks (daytime filter, Word rendering)
pydeck	0.9.2	Deck.gl map rendering
opentelemetry-sdk	>=1.20.0, <2.0.0	Observability/tracing

This workflow has migrated to the consolidated ecoscope-platform package scheme. Packages are distributed via the `repo.prefix.dev` conda channels and pinned to compatible version ranges. The runtime environment is managed by **pixi**.